



Built on the Zama FHE protocol

WRAPLINE

The Confidential Wrapper Registry, turned into a product.

Ethereum Sepolia + Mainnet · ERC-20 ↔ ERC-7984 · EIP-712 user-decryption

Built by @Sarnav07 · @vihaan1016

wrapline.vercel.app

THE PROBLEM WITH *the registry today*

Without a product layer

FRAGMENTED

Look-alike tokens everywhere

Every team ships against its own mock ERC-20s and wrappers instead of the canonical ones.

01

Duplicated tokens

Developers spin up their own testnet ERC-20s and ERC-7984 wrappers rather than reuse the official Zama Wrappers Registry.

02

Integrations don't compose

Each app targets a slightly different set of tokens, so confidential assets from one app don't interoperate with the next.

03

Wallets full of noise

Users end up holding confidential assets that only *look* like the real thing — the ecosystem fragments.

ONE REGISTRY, *every pair*

Wrapline is the missing product layer on the official registry — reading the on-chain Wrappers Registry as the source of truth, with a local overlay for dev-only pairs.

01

Browse

Every canonical ERC-20 ↔ ERC-7984 pair on Sepolia + Mainnet, with live metadata: name, symbol, decimals, addresses.

02

Wrap & unwrap

Convert any registry ERC-20 into its confidential twin and back — clean approvals, resumable async unwrap.

03

Decrypt any ERC-7984

Reveal your own balance of *any* confidential token via EIP-712 — registry or arbitrary paste-in.

04

Sepolia faucet

Claim the official cTokenMocks in one click and try the whole flow immediately.

Hybrid sourcing · on-chain primary + local overlay · deduped, on-chain always wins

LIVE DEMO

END-TO-END *on Sepolia*

Connect a wallet and run the full loop — six steps, no leaving the app.

01

Browse the registry

Every canonical ERC-20 ↔ ERC-7984 pair, live metadata, dual-chain.

03

Wrap a cTokenMock

Approve → encrypt → wrap 1:1 into its confidential twin.

05

Unwrap back

Async request → KMS decrypt → finalize; resumable.

02

Claim from the faucet

Mint an official cTokenMock on Sepolia in one click.

04

Decrypt the balance

Reveal the new ERC-7984 balance via EIP-712.

06

Decrypt any ERC-7984

Paste any confidential token — even outside the registry.

BROWSE *the registry*

Registry 18 pairs		All 18	Sepolia 9	Mainnet 9
● Sepolia	onchain	● Sepolia	onchain	
USDCMock → cUSDCMock		USDTMock → cUSDTMock		
USD Coin (Mock)		Tether USD (Mock)		
0x9b5C...DFfF · 0x7c5B...3639 · 6 dec		0xa7dA...e9b0 · 0x4E7B...4491 · 6 dec		
● Sepolia	onchain	● Sepolia	onchain	
WETHMock → cWETHMock		BRONMock → cBRONMock		
Wrapped Ether (Mock)		Bron (Mock)		
0xff54...5f3F · 0x4620...3158 · 18 dec		0xFf02...b25E · 0xaa56...C891 · 18 dec		
● Sepolia	onchain	● Sepolia	onchain	

All **8 official cTokenMocks** surfaced from the on-chain Wrappers Registry, plus a dual-chain filter — **Sepolia 9 · Mainnet 9**.

name · **symbol** · **decimals** ·

both addresses

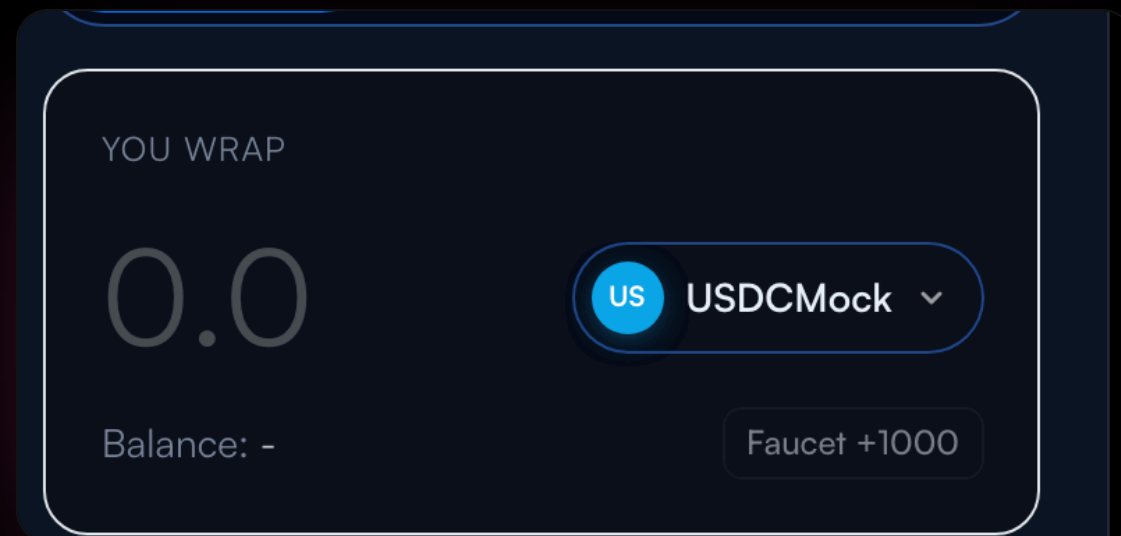
On-chain is the source of truth; a local overlay fills dev-only pairs, deduped.

CLAIM *from the faucet*

One click on **Faucet +1000** mints the official cTokenMock ERC-20 straight to your wallet on Sepolia.

mint → **balance updates** → **ready to wrap**

Per-address cap aware — the button disables when a claim would exceed the cap.



WRAP *a cTokenMock*

- 1 Approve ERC-20 allowance
- 2 FHE-encrypt the amount *client-side*
- 3 Wrap tx → confidential ERC-7984

Pre-flight guards catch missing allowance + insufficient balance before you spend gas. Wrapped **1:1** into cUSDCMock — only the ciphertext handle lands on-chain.

Wrap, unwrap, and decrypt
confidential tokens.

Wrap

Unwrap

Decrypt

YOU WRAP

0.0

us

USDCMock ▾

Balance: -

Faucet +1000



YOU RECEIVE

0.0

us

cUSDCMock

DECRYPT *the balance*

Wrap, unwrap, and decrypt
confidential tokens.

Wrap

Unwrap

Decrypt

Reveal your own ERC-7984 balance via EIP-712. The first reveal asks for one signature, then reuses the session.

Connect Wallet

YOUR TOKENS

Scan my balances

REGISTRY TOKEN

US

cUSDCMock ▾

US

cUSDCMock · 0x7c5B...3639

Reveal

.....

The freshly-minted ERC-7984 balance is revealed through the **EIP-712 user-decryption** flow.

Sign once → **session cached** → **cleartext balance**

Later reveals reuse the session — no repeat prompt.
The public only ever saw a ciphertext handle.

UNWRAP *back to ERC-20*

request → **KMS public-decrypt** → **finalize**

A three-step async flow: the plaintext amount is released by Zama's KMS before the contract returns the underlying ERC-20.

Every step is checkpointed to IndexedDB — **resumable**, even across devices, if you refresh mid-flow.

Wrap, unwrap, and decrypt confidential tokens.

Wrap

Unwrap

Decrypt

ERC-7984 → ERC-20. Async: the unwrap is requested on-chain, the KMS publicly decrypts the amount, then the wrapper finalizes. Reveal your balance below to see what you can unwrap.

Recover unwrap from another device...

YOU UNWRAP

0.0

US cUSDCMock ▾

cUSDCMock balance

-



DECRYPT *any* ERC-7984

ANY ERC-7984 ADDRESS

7c5BF43B851c1dffa4feE8dB225b87f2C223639

0x7c5B...3639

Reveal

.....

Paste **any** ERC-7984 address — a token that was never registered here — and reveal *your* balance the same way.

paste 0x... → **is-confidential?** → **reveal**

Auto-detect also scans the wallet for non-zero confidential holdings. Decryption isn't limited to the registry.

EXTENSIBILITY

ADD A *new pair*

Two documented paths — on-chain always wins over the local overlay via dedup.

A • On-chain registration

```
# picked up automatically as source: onchain
PRIVATE_KEY=0x... \
ERC20_ADDRESS=0x... \
WRAPPER_ADDRESS=0x... \
  bash scripts/register-pair.sh
```

B • Local overlay — config/pairs.ts

```
CUSTOM_PAIRS.push({
  chainId: sepolia.id,
  erc20Address: "0xYourErc20...",
  confidentialTokenAddress: "0xWrapper...",
  underlying: { name:"My Token", symbol:"MTK", decimals:18 },
  confidential: { name:"Conf MTK", symbol:"cMTK", decimals:18 },
})
```

Documented in the README with this exact example.

JUDGED *on*

01 Coverage

All 8 official cTokenMocks surfaced from on-chain; dual-chain Sepolia + Mainnet browse.

03 Extensibility

Two documented add-pair paths — on-chain script + local config — with a real example.

05 Code quality

Typed end-to-end, single error funnel, no debug residue, SSR-safe readiness gates.

02 Correctness

Wrap, unwrap, and EIP-712 user-decrypt produce the expected on-chain results.

04 UX

Approvals, network switching, resumable unwrap, and every error humanized — never a raw revert.

06 Production-readiness

Live on Sepolia + Mainnet, public-RPC fallback, Mainnet writes behind an explicit confirm.



Wrapline — confidential tokens, made usable.

THANK you

wrapline.vercel.app

Built by @Sarnav07 · @vihaan1016 · github.com/Sarnav07/Wrapline

Not affiliated with Zama. Built on the public Zama Protocol.